Lossless Compression Project: Issued Oct. 14, Due Oct. 29 at 5:00 p.m.

Overview

Natural language, such as English, is an important and fundamental data source. From an information theory standpoint, natural language is quite compressible; the entropy of English is at most 0.75 bits per character. The goal of this project is to design an encoder and decoder for compressing English.

We will focus on compressing a particular text file comprised of a concatenation of several novels by the early 19th-century author Jane Austen. The file consists of roughly 477,000 words and about two and a half million characters. The goal is to compress it as much as possible.

Your assignment is to write a Python script that, after reading from a data file that you create, reproduces this file in its entirety. We view the data file as the compressed version of the text and the Python script as the decoder (i.e., the decompressor). You may work in teams of at most two, and teams may straddle the ECE 5620/ECE 7620 courses.

Rules

You are asked to submit exactly three files: a data file, a decompression script, and a report. The data file must be named `compressed` and the decompression script must be named `decompress.py`.

We place no constraints whatsoever on the `compressed` file. You may place whatever data you like in this file in whatever format you like. The file `decompress.py`, on the other hand, must be a Python script that runs under the Python 3.12.4 installation on the ECE School's Linux cluster (`ecelinux.ece`), which is the version of Python that is run if you type `module load anaconda3` and then `python`.

Your `decompress.py` script may only import the `struct` and `decimal` modules. The script must create a text file named `out.txt` whose contents exactly matches the master file `austen.txt` that is posted on the course Canvas site. This can be tested using the UNIX command

```
diff out.txt austen.txt
```

The contents of the two files are identical if and only if this command returns no output.

Your decompression script may not use more than 20 minutes of CPU time in user mode on the Linux cluster, nor may it call external executables or import

 ©2026 A. B. Wagner

any modules other than the one listed above. You can determine how many CPU-seconds your script spends in user mode using the command

```
% module load anaconda3
% /usr/bin/time --format %U python decompress.py
```

We define the length of your compressed version of the text to be the *sum* of the file size of `compressed` and the size of your decompressor after it has been compiled to Python bytecode. This compilation can be accomplished with the Python interpreter via the commands

```
>>> import py_compile
>>> py_compile.compile('decompress.py')
```

This will create a file

```
__pycache__/decompress.pyc
```

consisting of the compiled bytecode. This *Kolmogorov complexity* approach (cf. Chapter 14 in Cover and Thomas) is necessary because one can embed information about the text in the Python script itself. In the same vein, you are not permitted to use multiple data files, or to call your data file anything other than `compressed`, because the names of the data files or the way that data is divided among them could be used to store some of the text. One could use the length of the uncompiled Python script instead of the length of the bytecode, but this would encourage overly concise programming that is difficult to read.

Submission

The project is due on Thursday, Oct. 29th at 5:00 p.m., with the prevailing policy on late submissions in effect.

You are asked to electronically submit your `compressed` and `decompress.py` files and your report as three separate files via the Canvas site. The report, which should be in PDF format, should describe your approach in sufficient detail to enable another student in the class to implement, *from scratch*, a solution that achieves similar performance to your submission. Please describe in particular any unique features of your solution.

Collaboration and AI Policy

You are encouraged to discuss this project with others, including current and former students in the course, to generate ideas. You are also encouraged to consult

 ©2026 A. B. Wagner

web resources and the published literature. You are welcome to use AI tools for coding, but *the report must be written by you and include a separate section titled "Acknowledgments" in which you provide attribution for all of the main ideas used in your solution and a description of what work was performed by you vs. what was performed by AI or drawn from external sources.*

Grading

30% of the overall score will be determined by the compression level of the solution. Solutions that do not correctly reproduce the original text file or that require more than 20 minutes of CPU time will receive little of this credit. 30% will be determined by the clarity and completeness of the written report. 30% will be determined by an oral code review that will take place after the deadline. The remaining 10% will be given for the acknowledgment section and adhering to the submission instructions described in this document.

There will be a prize for the team whose submission achieves the most compression among those submitted on-time, assuming there is only one such submission.

Tips

1. There is a sample solution, consisting of the two required files (and the Python script used to generated the `compressed` file), on the course webpage. It does not achieve anywhere near the maximum amount of compression. It is simply intended to get you started.

2. Since you need not submit a program that creates `compressed`, your encoding does not need to be done via a Python script that you write. For this you are welcome to use UNIX tools or programs written in other languages. You should describe how this file was created in your report, however. Also note that the 20-minute limit is on the *decoding* process. Encoding can take as long as you like.

3. The text file `austen.txt` is entirely lower case and has had some of its punctuation removed. This structure can be exploited for compression purposes.

4. The speed of convergence of the compression performance of universal compression algorithms to the entropy limit governs how well they compress finite-length strings. As we saw in class, algorithms that are universal with respect to a small class of distributions generally achieve faster convergence

 ©2026 A. B. Wagner

than those that are universal with respect to a larger class. This suggests tailoring the compression algorithm to the particular structure of the file. One can find general-purpose universal compressors written in Python on the Internet, but these are usually rather long programs that learn the structure of our file very slowly due to their generality. While you are welcome to incorporate these programs into your solution (as long as you properly acknowledge them and explain in detail in your report how they work), you will probably obtain better results by creating your own scheme that is tailored to this specific text. In past years, many students have obtained solutions that beat general purpose compressors such as `gzip` by a significant margin, even if one does not include the length of the `gzip` executable in the calculation.

5. Be aware that opening `austen.txt` in certain editors can result in changes to the whitespace in the file. The correct MD5 hash is:

   ```
   MD5 (austen.txt) = 967205af325fc85e5783b6847eaf8dba
   ```

6. If you choose to develop your solution on a machine other than the ECE School's Linux cluster or using a different Python interpreter, you should periodically verify that your solution runs correctly on the Linux cluster using the command above.

7. You might find it helpful to debug your code by first compressing a shortened version of `austen.txt`.

Good luck!

 ©2026 A. B. Wagner